CAPSTONE PRESENTATION - TRACK A

KijaniKiosk
Staging-to-Production Release Path

Dalton Wambui | v1.0.0 | feature/capstone-track-a

Repository: github.com/mwekundujohn-ship-it/kijaniosk-devops-foundation

One tested image. One approval gate. One evidence trail from commit to production.

PROBLEM AND SCOPE

A deploy is not a release until it is proven

Problem

- Weekly work existed, but releases were not one repeatable path.
- A green pipeline can still promote the wrong artifact.
- Production needs a human decision with evidence, not just a command.

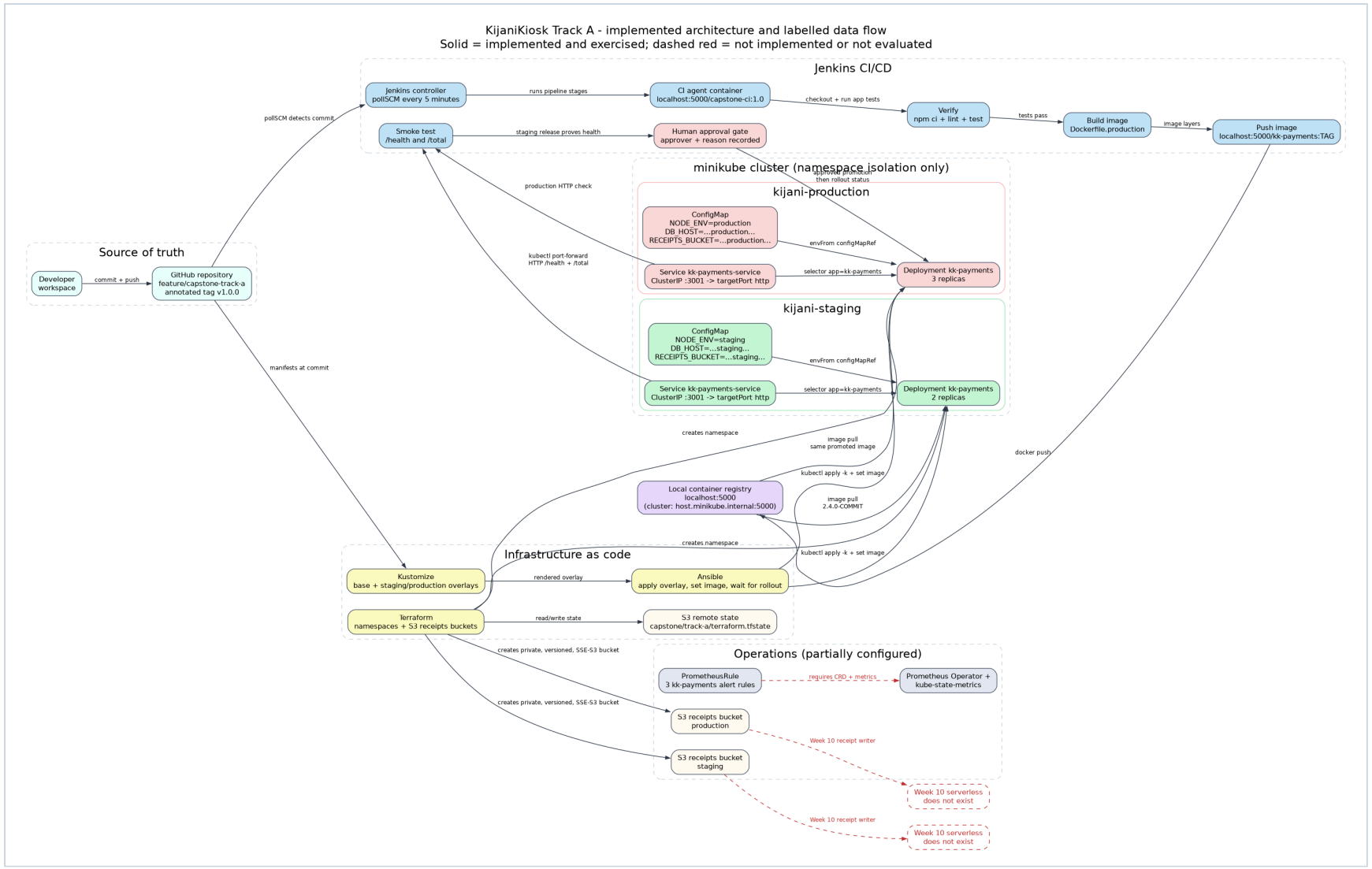
Track A deliverable

- Terraform environments and private receipt buckets.
- One Kustomize base, two environment overlays.
- Jenkins verifies, builds once, smoke tests staging, gates production, promotes the same image.
- Ansible applies and waits for rollout.

Stated boundary: Week 10 serverless receipts are not implemented. The bucket and environment key are prepared and the gap is not hidden.

ARCHITECTURE

Every solid arrow is implemented and exercised



Source: docs/architecture.dot; dashed red = not implemented or not evaluated

DELIVERY

The pipeline stops before the risk reaches production

Verify	Build once	Push	Staging	Smoke	Approve	Promote	Prove
npm ci, lint, tests	Dockerfile.production	local registry	apply, set image, rollout	health + version + total=42	approver + reason	same image tag	production smoke

Design rule: build once, promote the same tag. Production never rebuilds from a different commit.

Failure rule: staging smoke or version assertion fails before the approval gate. The operator never approves an unproven artifact.

INFRASTRUCTURE AND CONFIGURATION

Same workload, different environment facts

Terraform

Creates both namespaces and separate private, versioned, SSE-S3 receipt buckets. State lives in S3, not in Git.

Kustomize

One base Deployment and Service. Overlays change namespace, replicas, labels, image placeholder, and ConfigMap values.

Ansible

Applies the overlay, then sets the exact image tag and waits for rollout. Apply alone is not a release.

Key	Staging	Production
`NODE_ENV`	staging	production
`DB_HOST`	...staging...	...production...
`RECEIPTS_BUCKET`	staging bucket	production bucket
`APP_VERSION`	Not in ConfigMap: baked into the immutable image	

RUNTIME AND VERIFICATION

Kubernetes proves the process; the smoke test proves the release

Runtime controls

- 2 staging replicas, 3 production replicas.
- Non-root UID/GID 10001, read-only root filesystem, all capabilities dropped.
- Readiness and liveness probes on `/health`.
- Resource requests and limits.
- ClusterIP Service on 3001 targeting named port `http`.

What the smoke test asserts

- `status=ok` and `service=kk-payments`.
- The served version equals the image tag being promoted.
- `POST /total` returns exactly 42 for two synthetic line items.
- It polls during rollout so an old pod cannot make the test lie.

A health check answers "is something running?" The release check answers "is the thing we built serving?"

EVIDENCE

Build #2: the full path completed

Verified

- Staging rollout and smoke test passed for `2.4.0-2a84c81`.
- Human approval gate paused production.
- Production rollout and smoke test passed.
- Evidence archived by Jenkins.

Artefacts

- `capstone/evidence/jenkins-build-2/console.txt`
- `staging-state.txt`, `production-state.txt`
- `staging-configmap.txt`
- Live state: staging 2/2, production 3/3.

Demo adds: approval reason record, negative smoke-test failure, and pod self-healing. Text claims are paired with live commands, not only screenshots.

KEY DECISION AND CORRECTION

The release tag is an assertion, not a label

What I got wrong first

- The smoke test accepted any HTTP 200, so build #1 could pass against an old pod.
- Ansible `apply -k` could reset the image to the overlay placeholder after Jenkins pushed the real tag.
- `APP_VERSION` was treated as runtime config instead of immutable build metadata.

What changed

- Poll `/health` until the served version equals the image tag.
- `apply -k`, then `set image`, then `rollout status`.
- Version is a Docker build argument, not a ConfigMap key.
- Three issues tracked and closed with commit `2a84c81`.

"The command succeeded" and "the release is trustworthy" are different claims.

Corrections: docs/ai-governance-log.md; issues #12-#14

AI GOVERNANCE

AI assistance was reviewed, corrected, and recorded

AI produced

Drafts for the smoke test, Ansible tasks, and environment configuration.

AI got wrong

Accepted an old pod as healthy; let apply reset the image tag; put build metadata in a ConfigMap.

Human correction

Version assertion, explicit set-image, immutable version. Each correction has a commit and a live check.

Governance rule: no fabricated peer review, no invented build results, no screenshot without a command behind it. The eight-field log records the prompt, output, error, correction, evidence, and checklist item.

PRODUCTION GAPS AND NEXT STEPS

What this is not, and what I would build next

Honest gaps

- Production is a namespace in the same minikube cluster.
- Week 10 receipt chain is not implemented.
- PrometheusRule is validated but not evaluated.
- DB_HOST is not consumed by a live database.
- Pipeline uses pollSCM, not merge-to-main.
- Peer review is still a self-review fallback.

Next iteration

- Merge-to-main trigger with a PR gate.
- Receipt producer writing to the staging bucket.
- Prometheus Operator + kube-state-metrics and a fired alert.
- External secret manager and registry signing.
- Independent classmate review before submission.

The value of the capstone is the evidence chain, not the list of tools.